ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		С.А. Рогачев
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

ФС Поста

по курсу: ТЕОРИЯ ВЫЧИСЛИТЕЛЬНЫХ ПРОЦЕССОВ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Цель работы:

Целью лабораторной работы является разработка формальной системы Поста (FSp) для вычисления заданной арифметической функции 3x - y, где x и y — натуральные числа, а также реализация программы на языке высокого уровня (Python), эмулирующей вычисления в этой системе. Программа должна корректно обрабатывать входные данные, включая ошибочные, и выводить результаты вычислений в соответствии с заданными требованиями.

Постановка задачи:

1. **Построить формальную систему Поста (FSp)**, реализующую вычисление функции 3x - y. Формальная система включает:
    - Собственный алфавит A.
    - Множество переменных X.
    - Множество аксиом A1.
    - Конечное множество продукций R.
2. **Разработать программу на языке Python**, которая:
    - Считывает входные данные (A, X, A1, R) из файла.
    - Проверяет корректность входных данных (символы принадлежат алфавиту, переменные — множеству X).
    - Эмулирует процесс вывода в FSp, применяя правила к аксиоме до тех пор, пока это возможно.
    - Записывает в выходной файл результаты каждого шага (исходная строка, применяемое правило, результат).
    - Выводит на экран сообщение об успешном выполнении или об ошибке (например, использование символа вне алфавита или переменной вне X).
3. **Требования к входным данным**:
    - Все данные хранятся в одном файле в формате:

A={<элемент алфавита 1>, ..., <элемент алфавита N>};

X={<переменная 1>, ..., <переменная K>};

A1={<аксиома 1>, ..., <аксиома M>};

R={<правило 1>, <правило 2>, ..., <правило L>}

4. **Требования к выходным данным**:
    - В выходной файл записываются шаги вычисления: исходная строка, применяемое правило, результат.
    - На экран выводится сообщение: «Вычисление прошло успешно» или сообщение об ошибке.

Формальная система Поста:

Для функции 3x - y (вариант 39) определена следующая формальная система:

- **Алфавит**: A = {a, 0, 1}.
- **Множество переменных**: X = {P}.
- **Множество аксиом**: A1 = {a^x 0 1^y}, где x и y — натуральные числа (входные параметры, задаваемые конкретной аксиомой).
- **Множество продукций**:
    1. aP → 111P (заменяет один символ a на три символа 1, увеличивая счётчик результата).
    2. 101 → 0 (вычитает y, удаляя 1 из результата и из y).
    3. 0 → ε (удаляет разделитель 0, когда вычитание завершено).

Процесс вычисления:

- Начинаем с аксиомы a^x 0 1^y (x символов a, разделитель 0, y символов 1).
- Правило 1: Заменяем a на 111, повторяя x раз, получая 1^(3x) 0 1^y.
- Правило 2: Удаляем по одной 1 из результата и из y, пока не закончатся 1^y, получая 1^(3x-y) 0.
- Правило 3: Удаляем 0, получая 1^(3x-y).

Результатом является строка из (3x - y) символов 1, что соответствует значению функции.

Листинг программы на языке высокого уровня с комментариями:

```
import re

def main():
    # Читать входной файл
    try:
        with open('fs.txt', 'r', encoding='utf-8') as file:
            content = file.read()
    except FileNotFoundError:
        print("Ошибка: Файл fs.txt не найден.")
        return

    # Парсинг секций
    alphabet = parse_section(content, r'A\s*=\s*\{(.*?)\};')
    variables = parse_section(content, r'X\s*=\s*\{(.*?)\};')
    axioms = parse_section(content, r'A1\s*=\s*\{(.*?)\};')
    rules_str = re.search(r'R\s*=\s*\{(.*?)\};', content, re.DOTALL)
    if rules_str:
        # Удаляем запятые и пробелы в конце строк правил
        rules = [line.strip().rstrip(',') for line in rules_str.group(1).split('\n')
if '->' in line.strip()]
    else:
        print("Ошибка: Секция R не найдена.")
        return

    if not all([alphabet, variables, axioms, rules]):
        print("Ошибка: Не все секции найдены в файле.")
        return

    var_set = set(variables)
    alph_set = set(alphabet)

    # Отладочный вывод
    print("Алфавит:", alph_set)
    print("Переменные:", var_set)
    print("Аксиомы:", axioms)
    print("Правила:", rules)

    # Проверка входных данных на ошибки
    for ax in axioms:
        for c in ax:
            if c not in alph_set:
                print(f"Ошибка: Символ '{c}' в аксиоме '{ax}' не входит в алфавит
{alph_set}.")
                return
    for rule in rules:
        try:
            left, right = rule.split('->')
            left = left.strip()
            right = right.strip()
```

```
            print(f"Проверка правила: {rule} (левая часть: '{left}', правая часть:
'{right}')")
            for c in left:
                if c not in alph_set and c not in var_set:
                    print(f"Ошибка: Символ '{c}' в левой части правила '{rule}' не
входит в алфавит {alph_set} или переменные {var_set}.")
                    return
            for c in right:
                if c and c not in alph_set and c not in var_set:
                    print(f"Ошибка: Символ '{c}' в правой части правила '{rule}' не
входит в алфавит {alph_set} или переменные {var_set}.")
                    return
        except ValueError:
            print(f"Ошибка: Некорректный формат правила '{rule}'.")
            return

    # Симуляция для каждой аксиомы
    with open('result.txt', 'w', encoding='utf-8') as f:
        for axiom in axioms:
            current = axiom
            while True:
                applied = False
                for rule in rules:
                    left, right = rule.split('->')
                    left = left.strip()
                    right = right.strip()
                    # Построить parts
                    parts = []
                    for c in left:
                        if c in var_set:
                            parts.append(('var', c))
                        else:
                            parts.append(('fixed', c))
                    # Построить regex
                    group_idx = 1
                    var_groups = {}
                    pattern_parts = []
                    for typ, val in parts:
                        if typ == 'fixed':
                            pattern_parts.append(re.escape(val))
                        else:
                            pattern_parts.append('(.*)')
                            if val not in var_groups:
                                var_groups[val] = []
                            var_groups[val].append(group_idx)
                            group_idx += 1
                    pattern = ''.join(pattern_parts)
                    match = re.search(pattern, current)
                    if match:
                        # Проверить одинаковые значения для одинаковых переменных
                        sub = {}
                        consistent = True
                        for v, idxs in var_groups.items():
```

```
                            values = [match.group(idx) for idx in idxs]
                            if len(set(values)) != 1:
                                consistent = False
                                break
                            sub[v] = values[0]
                        if not consistent:
                            continue
                        # Построить right
                        right_build = ''
                        for c in right:
                            if c in var_set:
                                right_build += sub[c]
                            else:
                                right_build += c
                        # Заменить
                        start, end = match.span()
                        new_string = current[:start] + right_build + current[end:]
                        # Проверить символы в новой строке
                        for c in new_string:
                            if c not in alph_set:
                                print(f"Ошибка: В ходе вычисления произошла ошибка:
Символ '{c}' не входит в алфавит {alph_set}.")
                                return
                        # Записать
                        f.write(current + '\n')
                        f.write(rule + '\n')
                        f.write(new_string + '\n\n')
                        current = new_string
                        applied = True
                        break
                if not applied:
                    break

    print("Вычисление прошло успешно.")

def parse_section(content, pattern):
    match = re.search(pattern, content, re.DOTALL)
    if match:
        items = [item.strip() for item in match.group(1).split(',') if item.strip()]
        return items
    return []

if __name__ == "__main__":
    main()
```

Содержимое входного файла согласно заданию:

Корректные входные данные (x=2, y=1):

A={a,0,1};

X={P};

A1={aa01};

R={aP->111P,

101->0,

0->};

Корректные входные данные (x=3, y=2):

A={a,0,1};

X={P};

A1={aaa011};

R={aP->111P,

101->0,

0->};

Некорректные входные данные (символ не из алфавита):

A={a,0,1};

X={P};

A1={aa0b};

R={aP->111P,

101->0,

0->};

Некорректные входные данные (переменная не из X):

A={a,0,1};

X={P};

A1={aa01};

R={aQ->111Q,

101->0,

0->};

Пример результатов выполнения:

```
Алфавит: {'1', '0', 'a'}
Переменные: {'P'}
Аксиомы: ['aa01']
Правила: ['aP->111P', '101->0', '0->']
Проверка правила: aP->111P (левая часть: 'aP', правая часть: '111P')
Проверка правила: 101->0 (левая часть: '101', правая часть: '0')
Проверка правила: 0-> (левая часть: '0', правая часть: '')
Вычисление прошло успешно.
```

```
aa01
aP->111P
111a01

111a01
aP->111P
11111101

11111101
101->0
111110

111110
0->
11111
```

```
Алфавит: {'1', '0', 'a'}
Переменные: {'P'}
Аксиомы: ['aaa011']
Правила: ['aP->111P', '101->0', '0->']
Проверка правила: aP->111P (левая часть: 'aP', правая часть: '111P')
Проверка правила: 101->0 (левая часть: '101', правая часть: '0')
Проверка правила: 0-> (левая часть: '0', правая часть: '')
Вычисление прошло успешно.
```

```
aaa011
aP->111P
111aa011

111aa011
aP->111P
111111a011

111111a011
aP->111P
111111111011

111111111011
101->0
1111111101

1111111101
101->0
11111110

11111110
0->
1111111
```

```
Алфавит: {'a', '1', '0'}
Переменные: {'P'}
Аксиомы: ['aa0b']
Правила: ['aP->111P', '101->0', '0->']
Ошибка: Символ 'b' в аксиоме 'aa0b' не входит в алфавит {'a', '1', '0'}.
```

```
aaa011
aP->111P
111aa011

111aa011
aP->111P
111111a011

111111a011
aP->111P
111111111011

111111111011
101->0
1111111101

1111111101
101->0
11111110

11111110
0->
1111111
```

```
Алфавит: {'a', '0', '1'}
Переменные: {'P'}
Аксиомы: ['aa01']
Правила: ['aQ->111Q', '101->0', '0->']
Проверка правила: aQ->111Q (левая часть: 'aQ', правая часть: '111Q')
Ошибка: Символ 'Q' в левой части правила 'aQ->111Q' не входит в алфавит {'a', '0', '1'} или переменные {'P'}.
```

```
aaa011
aP->111P
111aa011

111aa011
aP->111P
111111a011

111111a011
aP->111P
111111111011

111111111011
101->0
1111111101

1111111101
101->0
11111110

11111110
0->
1111111
```

Вывод:

В ходе лабораторной работы была разработана формальная система Поста для вычисления функции 3x - y, включающая алфавит, переменные, аксиомы и правила вывода. Создана программа на Python, которая успешно эмулирует вычисления, обрабатывает входные данные из файла, корректно выявляет ошибки (например, символы вне алфавита или переменные вне X) и записывает результаты в выходной файл. Программа протестирована на корректных и некорректных входных данных, демонстрируя правильную обработку всех случаев. Результаты вычислений для корректных данных соответствуют ожидаемому значению функции 3x - y, а для некорректных данных выводятся соответствующие сообщения об ошибках.